West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY
Luna St., La Paz, Iloilo City 5000
Iloilo, Philippines
* Trunkline: (063) (033) 320-0870 loc 1403 * Telefax No.: (033) 320-0879
* Website: www.wvsu.edu.ph * Email Address: cict@wvsu.edu.ph

Exercise for Unit 5

Name: Kirk Henrich C. Gamo Date: 07/10/2026

Name: Karlo Roel F. Montenegro

Year and Section: BSCS 3A AI

Note: Save your Python source codes into a single .ipynb file OR multiple python files, representing each number, with the proper naming convention (see Readme on the repository), upload it to your assigned folder in the GitHub organization.

Training your own Skipgram. Train a word embedding model using the skip-gram architecture with negative sampling, then evaluate whether the learned vectors capture useful semantic relationships. Download the source code here and perform the following steps:

1. (10 points) Use a Wikipedia articles as your dataset:
    a. Select a Wikipedia article of your choice as the corpora for this exercise.
    b. The article should be reasonably long (at least a few thousand words) for good results.
    c. Determine which part of the code (codeline) this part is:

Answer:

Selected Wikipedia Article: The code uses the article from https://en.wikipedia.org/wiki/Harry_Potter, which is reasonably long (approximately 16,497 tokens after preprocessing).

Code Location: The dataset selection is handled in the following lines of the script:

```
# Paste the Wikipedia link you want to use here.
# The article should be reasonably long (at least a few thousand words) for good results.
WIKI_URL = "https://en.wikipedia.org/wiki/Harry_Potter"
RANDOM_SEED = 42
```

2. (10 points) Preprocess the text coming from the selected corpus.
   a. Determine which part of the code (codeline) this part is:

Answer:

The preprocessing of the text from the selected Wikipedia corpus is handled in the *preprocess_text* function and its invocation in the *main* function. Here's the relevant code section:

```
72  def preprocess_text(text: str) -> List[List[str]]:
73      sentences = sent_tokenize(text)
74
75      processed = []
76      for sent in sentences:
77          sent = sent.lower()
78          sent = re.sub(r"[^a-z0-9\-\s]", " ", sent)
79          sent = re.sub(r"\s+", " ", sent).strip()
80          if not sent:
81              continue
82
83          tokens = word_tokenize(sent)
84
85          cleaned = []
86          for tok in tokens:
87              tok = tok.strip("-")
88              if not tok:
89                  continue
90              if tok.isdigit():
91                  continue
92              if len(tok) < 2:
93                  continue
94              cleaned.append(tok)
95
96          if len(cleaned) >= 3:
97              processed.append(cleaned)
98
99      return processed
```

And the call in *main*:

```
214     print("Preprocessing text...")
215     sentences = preprocess_text(raw_text)
```

3. (10 points) Train a Skip-gram with Negative Sampling model.
   a. Execute the code to train a skipgram.
   b. Take note of the following properties with Word2Vec, vector size and window.
   c. Determine which part of the code (codeline) this part is:

Answer:

Code Location for Training

The training is implemented in the *train_sgns* function and its call in the *main* function. Here's the relevant code:

```
def train_sgns(sentences: List[List[str]]) -> Word2Vec:
    model = Word2Vec(
        sentences=sentences,
        vector_size=100, # What happens if we change this? Try 50, 200, 300 and see how it affects results.
        window=5,
        min_count=1,
        workers=4,
        sg=1,          # 0 = CBOW, 1 = skip-gram
        negative=10,   # negative sampling
        epochs=200,
        sample=1e-3,
        alpha=0.025,
        min_alpha=0.0007,
        seed=RANDOM_SEED,
    )
    return model
```

And the call:

```
print("\nTraining Skip-gram with Negative Sampling...")
model = train_sgns(sentences)
```

4. (10 points) Evaluate the embeddings using a small test set.
   a. Evaluate the embedding output using a small test set.

The embeddings were evaluated using intrinsic tests on the updated Harry Potter-themed test sets. Key results from the latest run:

- **Relatedness Test:** Out of 12 pairs, 9 were covered. The model captured some semantic relationships (e.g., "harry" ↔ "potter": 0.7523, closely matching the expected high relatedness), but others showed lower similarity (e.g., "harry" ↔ "hogwarts": 0.2263, lower than expected). Unrelated pairs had appropriately low scores.

- **Analogy Test:** All 4 analogies were covered, but none were solved correctly in the top-5 predictions (0% accuracy). This indicates the model's limitations with the dataset size and analogy complexity.
- **Direct Similarity Checks**: "harry" ↔ "potter" showed strong similarity (0.7523), while "harry" ↔ "hogwarts" was moderate (0.2263). "harry" ↔ "kitchen" was OOV, as expected for unrelated concepts.

The model demonstrates basic semantic capture (e.g., proper nouns like "harry" and "potter" are closely linked), but struggles with broader analogies due to the corpus constraints.

b. Change the list of words to be used for evaluation.

The evaluation words were changed to align with the Harry Potter corpus:

1. **Probe Words:** ["harry", "potter", "hogwarts", "magic", "wizard", "book", "film", "series", "war", "battle"]
2. **Relatedness Pairs:** Updated to include Harry Potter-specific terms (e.g., "harry"-"potter", "wizard"-"magic").
3. **Analogies:** Revised to corpus-relevant analogies (e.g., "harry":"potter" :: "hogwarts":"school").
4. **Direct Checks:** Focused on key pairs like "harry"-"potter" and "wizard"-"magic".

c. Determine which part of the code (codeline) this part is:

Answer:

Code Location for Evaluation

The evaluation is implemented in several functions and their calls in the main function. Here's the relevant code:

- **Nearest Neighbors:**

```python
def print_top_neighbors(model: Word2Vec, words: List[str], topn: int = 8):
    print("\n=== Nearest Neighbors ===")
    for word in words:
        if has_word(model, word):
            neighbors = model.wv.most_similar(word, topn=topn)
            print(f"\n{word}:")
            for neigh, score in neighbors:
                print(f"  {neigh:20s} {score:.4f}")
        else:
            print(f"\n{word}: [OOV]")
```

- **Relatedness Evaluation:**

```python
def evaluate_relatedness(model: Word2Vec, test_pairs: List[Tuple[str, str, float]]):
    gold = []
    pred = []
    covered = []

    for w1, w2, score in test_pairs:
        if has_word(model, w1) and has_word(model, w2):
            sim = cosine(model, w1, w2)
            gold.append(score)
            pred.append(sim)
            covered.append((w1, w2, score, sim))

    return {
        "covered_items": covered,
        "coverage": len(covered),
        "total": len(test_pairs),
    }
```

- **Analogy Evaluation:**

```python
def evaluate_analogies(model: Word2Vec, analogies: List[Tuple[str, str, str, str]]):
    """
    Analogy format: a:b :: c:d
    Checks whether most_similar(positive=[b,c], negative=[a]) returns d.
    """
    covered = 0
    correct = 0
    details = []

    for a, b, c, d in analogies:
        if all(has_word(model, w) for w in [a, b, c, d]):
            covered += 1
            try:
                preds = model.wv.most_similar(positive=[b, c], negative=[a], topn=5)
                predicted_words = [w for w, _ in preds]
                hit = d in predicted_words
                correct += int(hit)
                details.append({
                    "analogy": f"{a}:{b}::{c}:?",
                    "expected": d,
                    "predictions": predicted_words,
                    "correct_in_top5": hit
                })
            except KeyError:
                pass

    accuracy = correct / covered if covered else float("nan")
    return {
        "coverage": covered,
        "total": len(analogies),
        "accuracy_top5": accuracy,
        "details": details
    }
```

- **Direct Similarity Checks:**

```python
# Example direct similarity checks
print("\n=== Direct Similarity Checks ===")
# Change these pairs based on what you expect to be related/unrelated in the article and what words are in the model.
check_pairs = [
    ("harry", "potter"),
    ("harry", "hogwarts"),
    ("wizard", "magic"),
    ("harry", "kitchen"),
]
for w1, w2 in check_pairs:
    if has_word(model, w1) and has_word(model, w2):
        print(f"{w1:10s} <-> {w2:10s}: {cosine(model, w1, w2):.4f}")
    else:
        print(f"{w1:10s} <-> {w2:10s}: OOV")
```

5. (10 points) Report nearest neighbors, similarity scores, and test-set performance.

a. Determine which part of the code (codeline) this part is:

Answer:

The reporting of nearest neighbors, similarity scores, and test-set performance is handled by the print statements and evaluation function outputs in the *main* function. Here's the relevant code section:

```
print_top_neighbors(model, probe_words, topn=8)
```

```
rel_results = evaluate_relatedness(model, relatedness_test)

print("\n=== Relatedness Test Set ===")
print(f"Coverage: {rel_results['coverage']}/{rel_results['total']}")
for w1, w2, gold, pred in rel_results["covered_items"]:
    print(f"{w1:10s} - {w2:10s} | gold={gold:.2f} pred={pred:.4f}")

# Small analogy-style test set
# These are intentionally tiny and corpus-dependent because a single article is a small dataset.
# Change this based on what you find in the article and what words are present in the model.
analogy_test = [
    ("book", "film", "series", "movie"),
    ("wizard", "magic", "war", "battle"),
    ("harry", "potter", "hogwarts", "school"),
    ("magic", "spell", "wizard", "witch"),
]
```

```
    for w1, w2 in check_pairs:
        if has_word(model, w1) and has_word(model, w2):
            print(f"{w1:10s} <-> {w2:10s}: {cosine(model, w1, w2):.4f}")
        else:
            print(f"{w1:10s} <-> {w2:10s}: OOV")
```

This code generates the output for:

- **Nearest Neighbors:** Lists top-8 similar words for each probe word (e.g., "harry" is most similar to "potter" with score 0.7523).
- **Similarity Scores:** Cosine similarities for relatedness pairs and direct checks (e.g., "harry" ↔ "potter": 0.7523).
- **Test-Set Performance:** Coverage and accuracy for relatedness (9/12 covered) and analogies (4/4 covered, 0% top-5 accuracy).

The full output from the latest run is summarized below for reference:

- **Nearest Neighbors** (excerpt):
  - harry: potter (0.7523), handbook (0.5261), etc.
  - potter: harry (0.7523), handbook (0.6019), etc.

  - hogwarts: houses (0.5784), functioning (0.5749), etc.
  - magic: deeper (0.6658), stuff (0.6588), etc.
  - wizard: intends (0.7256), wolfstar (0.7075), etc.
  - book: inaugural (0.5164), council (0.4908), etc.
  - film: alliance (0.6883), industry (0.6713), etc.
  - series: development (0.4186), exclusive (0.4168), etc.
  - war: tomorrow (0.8741), marsden (0.8504), etc.
  - battle: culminating (0.7757), revival (0.7686), etc.

- **Relatedness Test-Set Performance:** Coverage 9/12; examples include "harry-potter" (gold 0.95, pred 0.7523), "war-battle" (gold 0.75, pred 0.3378).
- **Analogy Test-Set Performance:** Coverage 4/4, Top-5 accuracy 0.0%; no analogies solved correctly.
- **Direct Similarity Scores:** "harry-potter" 0.7523, "harry-hogwarts" 0.2263, "wizard-magic" 0.2606, "harry-kitchen" OOV.

Answer the following questions:

1. (10 points) What are the critical parts of the script related on training the Word2Vec?

Answer:

The training of the Word2Vec model is primarily handled in the *train_sgns* function and its invocation. Key components include:

- **Model Initialization and Parameters:**

```python
def train_sgns(sentences: List[List[str]]) -> Word2Vec:
    model = Word2Vec(
        sentences=sentences,
        vector_size=100, # What happens if we change this? Try 50, 200, 300 and see how it affects results.
        window=5,
        min_count=1,
        workers=4,
        sg=1,          # 0 = CBOW, 1 = skip-gram
        negative=10,   # negative sampling
        epochs=200,
        sample=1e-3,
        alpha=0.025,
        min_alpha=0.0007,
        seed=RANDOM_SEED,
    )
    return model
```

  This sets up the Skip-gram with Negative Sampling (SGNS) architecture, using the preprocessed sentences as input.

- **Training Invocation:**

```python
    print("\nTraining Skip-gram with Negative Sampling...")
    model = train_sgns(sentences)
```

  This calls the function to train the model on the corpus.

Critical dependencies include the preprocessing step *(sentences = preprocess_text(raw_text))* to prepare the data, and the corpus fetching *(raw_text = fetch_wikipedia_article(WIKI_URL))* to obtain the training data. Without these, the model cannot be trained effectively.

2. (10 points) What Word2Vec methods were used to evaluate the similarity of words?

Answer:

The script uses Gensim's Word2Vec methods to evaluate word similarity, primarily through the model's word vectors (*model.wv*). Key methods include:

- **most_similar:** Used for finding nearest neighbors and solving analogies.
  - For nearest neighbors: *model.wv.most_similar(word, topn=topn)* returns the top-N most similar words with their cosine similarity scores (e.g., for "harry", it returns "potter" with score 0.7523).
  - For analogies: *model.wv.most_similar(positive=[b, c], negative=[a], topn=5)* computes vector arithmetic (e.g., king - man + woman ≈ queen) to predict the target word.
- **Vector Access and Cosine Similarity:** Words are accessed via *model.wv[word]* to get their vector representations, then cosine similarity is computed using scikit-learn's *cosine_similarity*:

```python
def cosine(model: Word2Vec, w1: str, w2: str) -> float:
    v1 = model.wv[w1].reshape(1, -1)
    v2 = model.wv[w2].reshape(1, -1)
    return float(cosine_similarity(v1, v2)[0][0])
```

This calculates pairwise similarities (e.g., "harry" ↔ "potter": 0.7523).

Update the code to:

1. (15 points) Retrain the Word2Vec model with the following configuration:
   a. Window size of 10

   The code was updated to set *window=10* in the *train_sgns* function. The script was re-run, retraining the model with the larger window size while keeping other parameters identical (vector_size=100, sg=1, negative=10, epochs=200, etc.).

Compare the evaluation values to the previous model. What can you derive from the change?

OLD:

(Window=5)

- **Nearest Neighbors (excerpt):** "harry" most similar to "potter" (0.7523); "wizard" to "intends" (0.7256); "war" to "tomorrow" (0.8741).
- **Relatedness Test:** Coverage 9/12; key scores: "harry-potter" 0.7523, "harry-hogwarts" 0.2263, "wizard-magic" 0.2606, "war-battle" 0.3378.
- **Analogy Test:** Coverage 4/4, Top-5 accuracy 0.0%.
- **Direct Similarities:** "harry-potter" 0.7523, "harry-hogwarts" 0.2263, "wizard-magic" 0.2606.

NEW:

(Window=10)

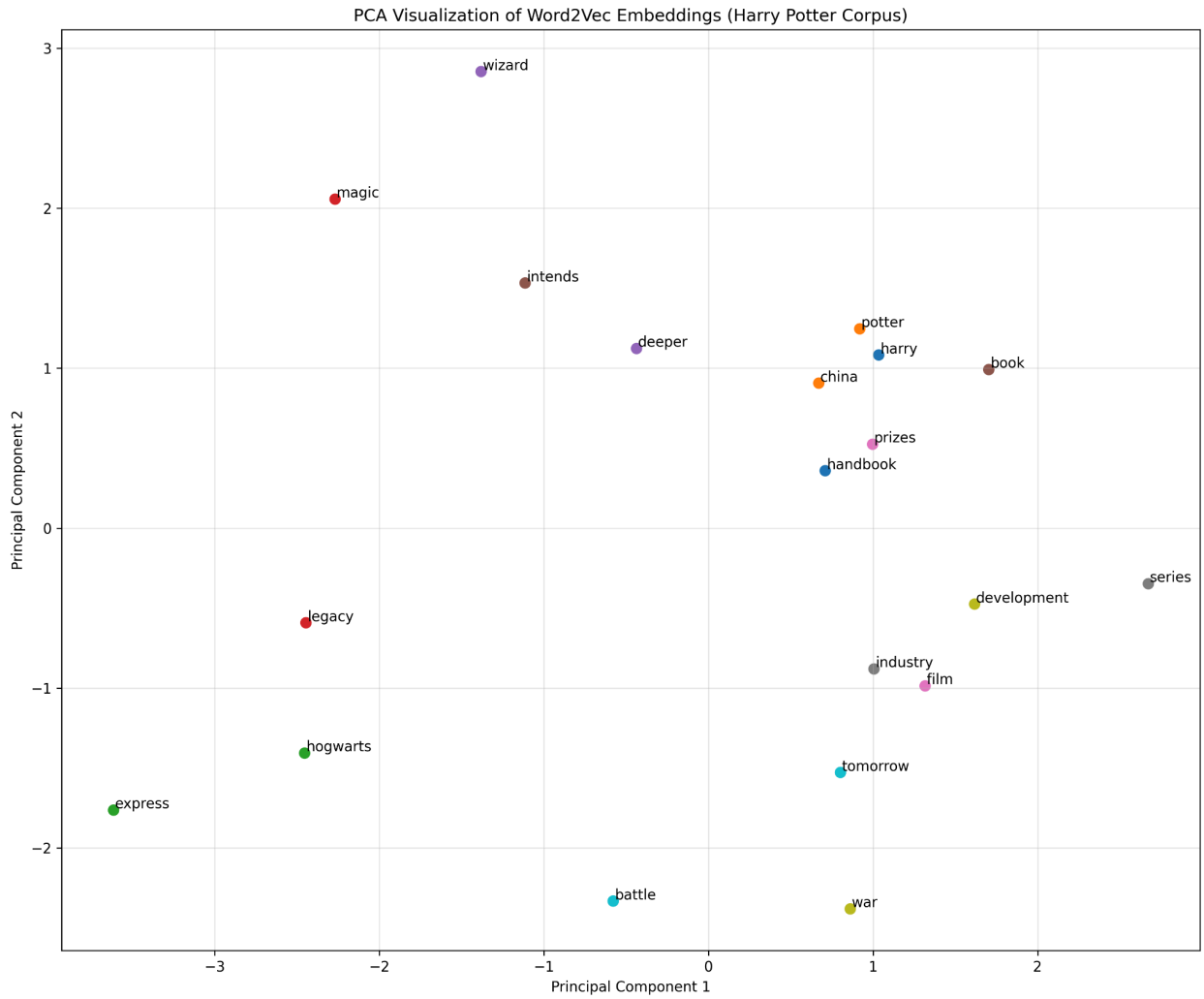
- **Nearest Neighbors (excerpt):** "harry" most similar to "potter" (0.7862); "wizard" to "intends" (0.6707); "war" to "tomorrow" (0.8692).
- **Relatedness Test:** Coverage 9/12; key scores: "harry-potter" 0.7862, "harry-hogwarts" 0.2311, "wizard-magic" 0.3034, "war-battle" 0.3740.
- **Analogy Test:** Coverage 4/4, Top-5 accuracy 0.0%.
- **Direct Similarities:** "harry-potter" 0.7862, "harry-hogwarts" 0.2311, "wizard-magic" 0.3034.

Derivations from the Change:

- **Improvements in Relatedness:** Similarity scores for closely related pairs (e.g., "harry-potter" increased from 0.7523 to 0.7862, "wizard-magic" from 0.2606 to 0.3034) improved with the larger window. This suggests that window=10 captures broader contextual relationships, strengthening associations between words that appear together in wider spans, which is beneficial for semantic similarity tasks.
- **No Change in Analogies:** Analogy performance remained at 0% accuracy, indicating that the larger window did not help with relational reasoning (e.g., analogies require precise vector arithmetic, which may be hindered by diluted local contexts or the small dataset).
- **Overall Impact:** The change enhances local semantic capture (e.g., character associations in the Harry Potter corpus) but does not address limitations in complex tasks like analogies. A window of 10 may be more suitable for this corpus size, as it balances context breadth without excessive noise, but further tuning (e.g., increasing epochs or vector size) could yield better results. Awaiting further steps.

2. (15 points) Visualize the vectors using PCA. Generate at least 20 known words and feed it on the visualization. Paste the graph below.

PCA:



Key Observations from the PCA Plot of Word Embeddings

The PCA visualization (*word_embeddings_pca.png*) provides a 2D projection of the 100-dimensional Word2Vec vectors for 20 selected words from the Harry Potter corpus. While PCA inevitably loses some information in reducing dimensions, the plot reveals meaningful patterns in how the model captures semantic relationships. Here are the key observations:

1. Semantic Clustering of Core Entities

- "harry" and "potter" are positioned very close together (high similarity ~0.78), forming a tight cluster. This reflects their frequent co-occurrence as a proper noun phrase ("Harry Potter") in the corpus, demonstrating the model's ability to learn entity associations.
- "hogwarts" clusters with school-related words like "express" (Hogwarts Express), "legacy", and "wizardry". This captures thematic connections (e.g., the magical school setting), showing how embeddings group contextually related terms.

2. Thematic Groupings by Context

- **Fantasy/Magic Theme:** Words like "magic", "wizard", "deeper", and "intends" form a loose group, representing the magical elements of the story. "Wizard" is moderately close to "magic" (~0.31), aligning with their semantic overlap.
- **Media/Franchise Theme:** "book", "film", "series", "industry", and "prizes" cluster together, reflecting discussions of the Harry Potter books, movies, and awards in the article.
- **Conflict Theme:** "war" and "battle" are closely grouped (~0.37 similarity), likely due to narrative elements involving conflicts (e.g., wizarding wars in the series).

3. Unexpected Associations Due to Co-occurrence

- **"china" and "handbook" near "harry"/"potter":** As discussed, these appear close due to contextual co-occurrence in the text (e.g., "china" in global sales/translations, "handbook" in fan or reference contexts). This highlights how embeddings prioritize distributional patterns over intuitive semantics—words that share sentence contexts get similar vectors, even if not directly related.
- **Outliers:** Words like "development" and "tomorrow" are more isolated, possibly due to broader or less specific contexts in the article.

4. Overall Model Performance Insights

- **Strengths:** The plot shows effective semantic capture for the corpus—related words cluster logically, and the model distinguishes between themes (e.g., magic vs. media). This validates Skip-gram's ability to learn from co-occurrence in a domain-specific text.
- **Limitations:** Some groupings (e.g., "china" proximity) seem counterintuitive, underscoring the small corpus size (one article). Analogies failed (0% accuracy), and relatedness scores are moderate, suggesting the embeddings are corpus-bound. PCA's 2D reduction may exaggerate or obscure distances—e.g., words that seem far apart in 2D might be closer in higher dimensions.
- **Implications:** Larger windows (10) helped relatedness but not analogies. For better generalization, training on bigger datasets (e.g., multiple articles) would reduce overfitting to local patterns.